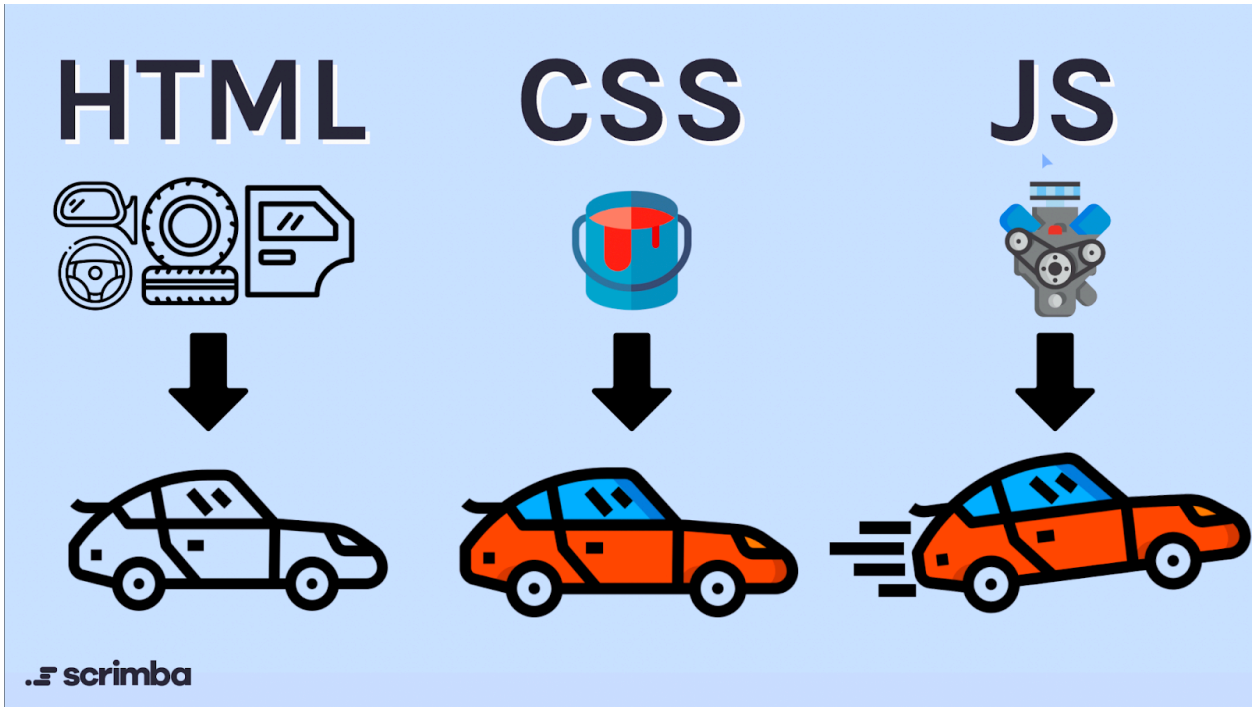
Java Script ⇒ .js ⇒ .ts

It is Programming scripting language

It give interactive ness or Functionalities of Web Pages

Java script is a high level programming scripting language and it's give more interactiveness of the web pages and web application.

We create or develop a static web application using only html,css when will you add your scripting concept of js that time only project will transforming to Dynamic web application



Importing methods 😎

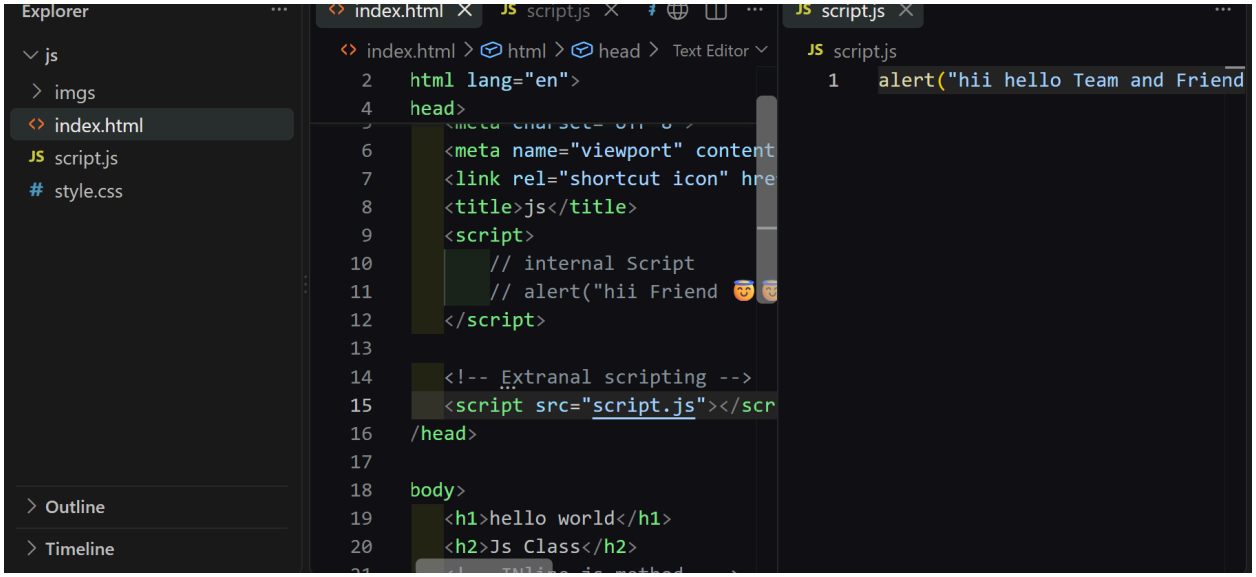
Inline Scripting method

```
<body>
    <h1>hello world</h1>
    <h2>Js Class</h2>
    <!-- INline js method  -->
    <script>
        alert("hii team")
    </script>
</body>
```

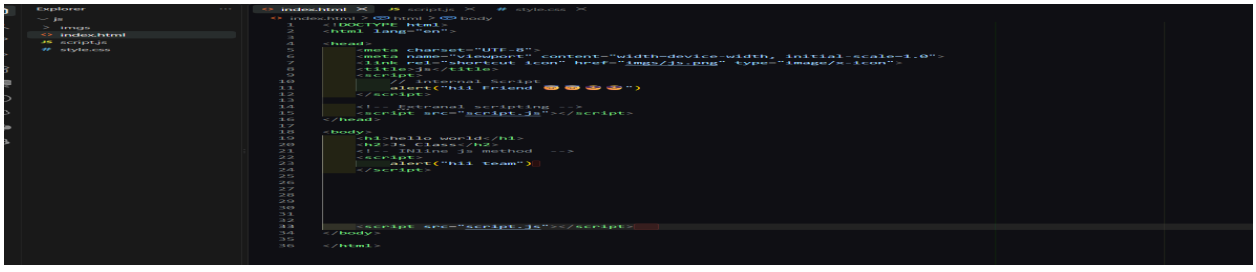
Internal Scripting method

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="shortcut icon" href="imgs/js.png" type="image/x-icon">
    <title>js</title>
    <script>
        // internal Script
        alert("hii Friend 😇😇😎😎")
    </script>
</head>
```

External Scripting method



LExical Scripting method



Javascript Vs Java 👍

Java	Javascript
Core Language	Scripting language
Combination of OOPS & DSA	Optional ⇒ OOPS & DSA
Structure & Syntax or Rules we wont avoid or overcome	Structure & Syntax or Rules we will or can avoid or overcome
Backend Logics	Frontend Logic or validation
Java ⇒ oops dsa ⇒ core concepts	ECMA script ⇒ similar to java ⇒ Js Js ⇒ ECMA script 5(ES 5) , ECMA script 6 (ES 6) ⇒ es6 (js)

JS Concepts 👍

ES 5 Topics

Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
Data Type ⇒ 2 +1

Primitive Data 7	Non primitive Data 3
Number String Boolean Bigint Undefined Null Symboles	Array ⇒ =[], ⇒ = [‘ ‘,””,1,true] Object ⇒ ={}, ⇒ ={ key : value } Function

```
<script>
        //  Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Data Type  ⇒ 2 +1
        // Stractured Data type  ==> Primitive Data type (7)
        // Number  ==> [0,1,2,3,4.......n] ==> without using any
symblole  "",''
        let num1 = 100;
        const Total = 2000;
        let myNum = 10.80;
        console.log(num1);
        console.log(Total);
```

```
        console.log(myNum);
        // String ==> [a,b,c,d......z] ==> with using any symbole like "
",''
        let myName = " ganapathi";
        let myName1 = 'raja';
        console.log(myName);
        console.log(myName1);
        // Boolean ==> [true, false ==> 0,1]
        // let isValid = true;
        let isValid = false;
        console.log(isValid);
        // bigINt [ (2n*3) ==> more the 5 digits ]
        let myMobile = 75202132895
        let myValue = 100000000000000000000000
        BigInt(myMobile);
        BigInt(myValue);
        console.log(BigInt(myValue))
        // Undefined ==> [ ? ] ==> may be getting a values
        var a = 1;
        console.log(a);
        // Null ==> [ { } ]  ==> never getting a values
        var a = null;
        // symbole [ @,!,#,$,%];

        // UnStractured Data type  ==> NON - Primitive Data type (3)
    </script>
```

```
<script>

        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Array  ==> arr or [ '',"",10]
        // Hytrogenius element an dstore continuos=use loaction
        // it has two important key words
        //  index ==>start with 0==>  addresss ==> position ==> refernce
key,
        //  element ==> values ==> datas ==>
        var myArr = [100, 200, 300, "ganapathi", "sunder", "hari", true,
false, 100.200, "%^$", 7502132895, "pc", "hello", '1'];
```

```
        console.log(myArr);
        console.log(myArr[0]); // index finding
        console.log(myArr[12]);
        // console.log(myArr[13]);
        // console.log(myArr[8]);
        // console.log(myArr[8, 9, 10]);

        // Object  ==> Stand alone entity  ==> obj or  = {} or ={key :
value}

        var myCar = {
            name: "spressoo",
            color: "grey",
            price: 500000
        };

        var myCar1 = {
            name: "spressoo",
            color: "grey",
            price: 500000,
            quality: function functionality() {
                console.log(myCar1.name);
                console.log(this.color); // 3rd calling object value ==>
this lkeyword to cll inside object within the function
            }
        };
        console.log(myCar);
        console.log(myCar.name); //1st method ==>  object litral method
        console.log(myCar.color);
        console.log(`${myCar.price}`); //2nd method ==>  object
destructing method

        console.log(myCar1.quality())
        //  console.log(${ myCar.price }); // object destructing method

        // Function  ==> fnction ==> funName () { return } ==>
=()=>{return}
    </script>
```

```
<script>
        //  Array  ==> = [ ]

        let myDatas = [1, 2, 3, 4, 5, "a", "b", "c", true, false, null,
7502132896,];
        console.log(myDatas);
        console.log(myDatas[0]);
        console.log(myDatas[1]);
        console.log(myDatas[5]);

        console.log(myDatas.indexOf(true));

        // Object ==> ={ key : value  }

        const car = {
            name: "bmw",
            color: "navy",
            price: 1200000,
            model: "m5"
        }

        console.log(car)
        console.log(car.model)
        console.log(car.name) // . 1st method litral

        console.log(`${car.price}`)
        console.log(`${car.color}`) // `${}` 2nd method object
desctructing

        let waterBottle = {
            name: "jio",
            qty: "5l",
            price: 20,
            capacity: function size() {
                let smallSize = "250ml";
                let sample = this.name
                console.log(smallSize)
                console.log(waterBottle.qty)
                console.log(this.name) // this 3rd method (.this) method
```

```
                console.log(this.price) // this measn value assigner
            }
        };
        console.log(waterBottle)
        console.log(waterBottle.capacity())
        // console.log(this.AnimationTimeline) // not adviceable

        // Functions  ==> (){} or =()=>{} ==> or function
        // Reusable code (organized) collection to perform single loci
operations
        // function funName(){ return } or const funNAme =()=>{return};

        // console.log(100 + 200);



        // function ==> 2 classification
        // with Parameter function
        // parameter ==> arguments ==> reference datas  ==> props ==>
params
        function sumOftwoNumbers(a, b) { // a,b parameter ==> value
carrier
            let c = a + b;
            return c;
        }

        const sumOfNumbers = (value1, value2) => {
            return valuesAdd = value1 + value2;
        }

        console.log("nasika 100,200:  ", sumOftwoNumbers(100, 200));
        console.log("pavidra 1000,2000:  ", sumOftwoNumbers(1000,
2000));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 20));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 1));

        // without  Parameter function

        function Nasika() {
            let a = 100;
```

```
            var b = 300;
            const c = a + b;
            return c;
            console.log(c)
        };
        Nasika();
        console.log(Nasika());

        function Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        };
        Pavidra()

        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())


        // functions 4 types
        // Function expression method
        // function FunName () { return }
        function Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        };
        Pavidra()
        // Arrow method ==> HOF ==> Call back ==> recurssive function
==> cohansive function ==>
        // variabledeg funcName =()=>{ return }
        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())
```

```
        // IIFE
        const myMainfun = (
            function Pavidra() {
                let x = 200, y = 300;
                const z = x + y;
                console.log(z);
            }
        )();
        // Generative Method
            function* Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        }

    </script>
```

Variable ⇒ Containers (temp storage) ⇒ 3 ⇒ var ,let, const
Variable Scoping & Hoisting ⇒ { } ⇒ 3

```
<script>

        // Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Collection of Inforamtaions or detalis
        // Raw facts ==> 0,1 ==> true  false
        // Data type ==> 2
        // Stractured Data type  ==> Primitive Data type (7)
        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Variable  ⇒ Containers (temp storage ) ⇒ 3 ⇒ var ,let,
const
        // var let const
        // To store temporarly in data in Opertaion basis

        // a = "ganapathi";

        // var a = "ganapathi";
        // var a = "sundar";
```

```
        // let a = "Keerthi";
        // let a = 1000;

        // const pi = 3.14
        // const Opi = 22 / 7

        // let pi = 20;

        // const a = 100;
        // const a = "ganapathi";

        // variable  varName = values

        var obj1 = "hii"
        let obj2 = 'hello'
        const obj3 = 100;

        // Variable  ⇒ {  } == > Scoping => 3
        // global scope  ==> ES5 scope  ==> var let const
        // local  scope ==>  Block level scope ==> functional scope ==>
let const
        // lexical  scope  ==> let const


        var trainer = "ganapathi";
        function batch14() {  // Globle scope
            console.log("hii globel", trainer);
            let trainee = "pavidra";
            console.log(trainee);

            function team1() { // local scope
                console.log(trainer); // lexivcaln
                console.log(trainee);
                const trainee1 = "Mani"
                console.log(trainee1)
            }
            team1()
            console.log(trainee1)
```

```
    }

    console.log(trainer);
    batch14();
    // console.log(trainee);


    function sla() {
        var trainer = "ganapathi";
        function batch14() {
            console.log(trainer);
            let trainee = "pavidra";
            function team1() {
                console.log(trainer);
                console.log(trainee);
                const trainee1 = "Mani"
                console.log(trainee1)
            }
        }
    }

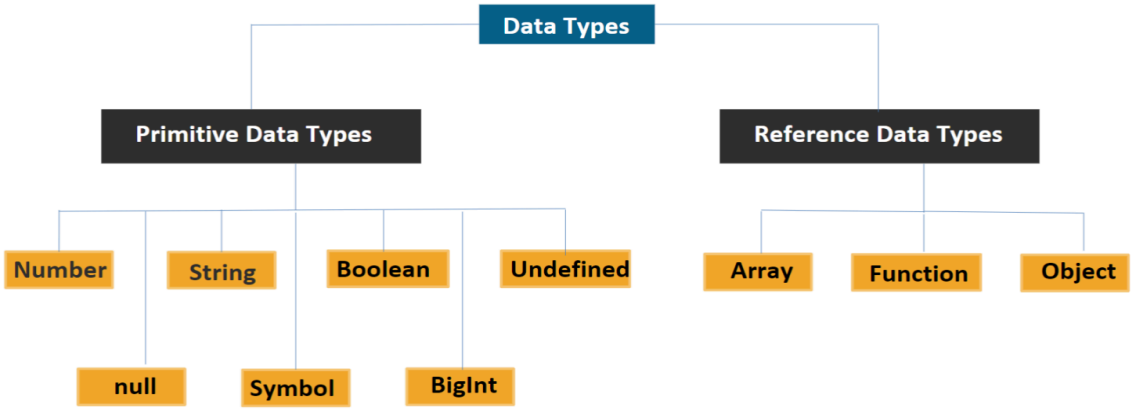
</script>
```

Var	Let	Const
Es5 version variable	Es6 version variable	Es6 version variable
Void scoping (public)	Block / functional scoping	Block / functional scoping
Global scoping variable	Local scoping variable	Local scoping variable
Read / write operation can perform	Read / write operation can perform	Read operation can perform
Changeable / Mutable	Changeable / Mutable	Non Changeable / imMutable
Scoping means ⇒ { }		
Variable Hoisting ⇒ var		

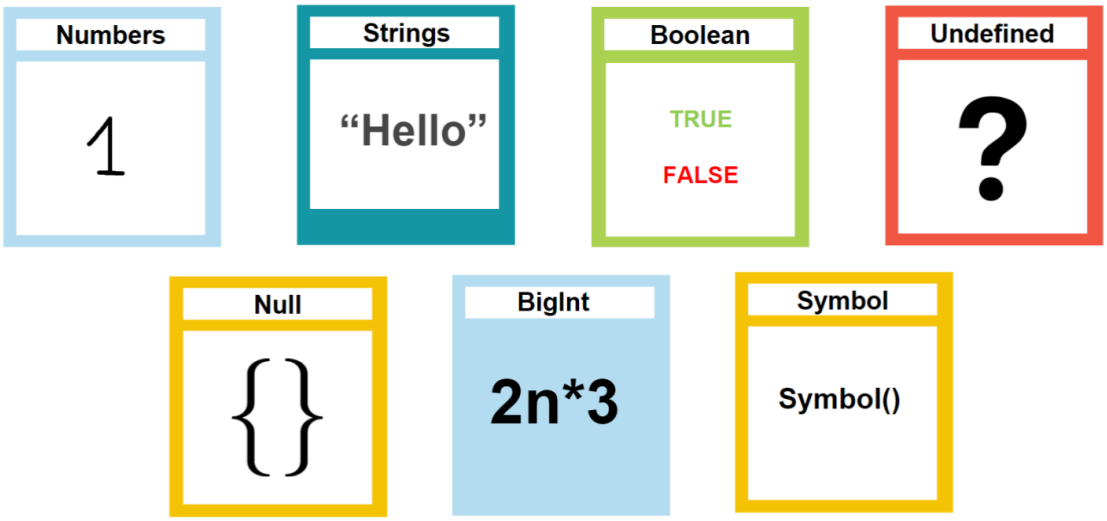
Web hoisting ⇒ Local project will migrate globalize project

```
// Variable Hoisting
        // const a = "Ganpathi";
        // let a = "Ganpathi";
        // var a = "Ganpathi";
        console.log(a);
        var a = "Ganpathi Raja";
        // let a = "Ganpathi Raja";
        // const a = "Ganpathi Raja";
```

Datatypes in JavaScript



Primitive Data Types



Reference Data Types

Objects: An object is a standalone entity. It consists of properties and methods.

```
Syntax: let obj1 = { }
```

Arrays: An Array is a collection of heterogenous elements which are stored at contiguous memory locations.

```
Syntax: let arr[ ] ;
let arr = new Array( )
```

Functions: A function is a block of organized, reusable code that is used to perform a single, related actions.

```
Syntax: function
name() {
alert("This is a
function");
}
```

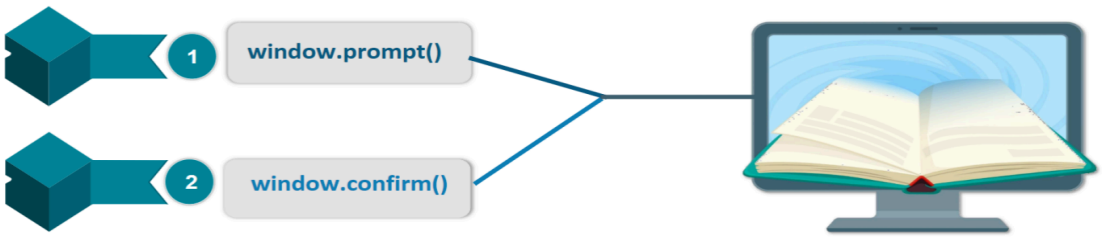
Understanding JavaScript Output Statements

- In JavaScript, there are different ways to display the data.
- Some of the commonly used ways to display output are:

Statement	Description
innerHTML	It writes a data into a certain HTML element
document.write()	It displays a data in the HTML output
window.alert()	It displays the data in the alert box
console.log()	It displays the data in the browser console

Understanding JavaScript Input Statements

- It is important to take input from the user to display the data as per their need.
- Apart from forms, JavaScript provides two statements to accept input from the user as mentioned below:



```
 <h2>Output methods</h2>

    <button onclick="outputMethod()">click me see my output</button>

    <h2 id="outputmethod">hii output methods</h2>

    <button onclick="outInnerhtmlmethod()">click me InnerHtml</button>


    <div>
        <h2 id="input1"></h2>
        <h2 id="input2"></h2>
        <button onclick="inputMethod1()">click Give input 1</button>
        <button onclick="inputMethod2()">click Give input 2</button>
    </div>
    <script>
        // Inputs 2 /
        // window.prompt();
        // window.prompt("Enter your thougt today ...!")
        function inputMethod1() {
            const userMsg = window.prompt("Enter your thougt today ...!");
            let myThought = document.getElementById("input1");
            myThought.innerHTML = userMsg;
        };
        // window.confirm();
        function inputMethod2() {
            if (window.confirm("Are you sure submit your form..👍👍!")) {
                let myinput2method = "successfull submitted your
form...👍👍👍"
                let myThought = document.getElementById("input2");
                myThought.innerHTML = myinput2method;
            }
            else {
                let myMessages = "Your Application not submitted ..sorry
....❌❌🤦‍♂️🤦‍♂️";
                let myThought = document.getElementById("input2");
                myThought.innerHTML = myMessages;
            }

        };
```

```
    //  Output methods 4
    // console.log(); ==> 4
    const myName = "Ganapathi Raja ";
    console.log(myName);
    console.info(myName);
    console.warn(myName);
    console.error(myName);


    // window.alert() ==> 2
    // window.alert(myName);
    // alert(myName);


    // document.write()
    document.write(myName);
    function outputMethod() {
        let messages = "hii team and Friends...😍😎😎😇😂";
        // document.write(messages);
        document.writeln(messages);
    };
    // innerHTML
    const outInnerhtmlmethod = () => {
        let messages = "hii team and Friends...😍😎😎😇😂";
        let myPlacehtml = document.getElementById("outputmethod");
        myPlacehtml.innerHTML = messages;
    }
</script>
```

Operators ⇒ 4 + 2
Z = x+y ⇒ let z = a+b;
Operands ⇒ x,y,z
Operators ⇒ +.=
OPeration ⇒ addition
Variable ⇒ let ⇒ temp

Operators	Symbols or Operators
Arithmetic Operators (7)	+,-,*,/,%,++,--
Assignment Operators (8)	=,=+,=-=,*,=/,=%,
Comparison Operators	==,===,!=,<,=<,>,>=
Logical Operators (3)	AND &&,OR ||,NOT !
Ternary ⇒ ?	? ⇒ if statements
typeOf ⇒ Operators	typeOf()

```
<script>



        // Arithmetic Operators (7)
        // +,-,*,/,%,++,--
        let a = 10; b = 30;
        const key = 1;
        // const key = 2;
        // const key = 3;
        // const key = 4;
        // const key = 5;
        // const key = 6;
        // const key = 7;
        // const key = 17;
        switch (key) {
            case 1:
                var c = a + b;
                console.log(c);
                break;

            case 2:
                var c = a - b;
                console.log(c);
                break;
            case 3:
                var c = a * b;
                console.log(c);
```

```
            break;
        case 4:
            var c = a / b;
            console.log(c);
            break;
        case 5:
            var c = a % b;
            console.log(c);
            break;
        case 6:
            a++
            var c = a + b;
            console.log(c);
            break;
        case 7:
            a--;
            var c = a + b;
            console.log(c);
            break;
        default:
            console.log("You enter invalid Operation")
            break;
    }


    // let x = 200; y = 400; z = x + y;
    // console.log(z);

    // console.log(10 + 20);

    // Assignment Operators (8)
    // =,=+,=-=,*,=/,=%,

    let x = 200;
    let y = 400;
    // x++;
    x = x + 40;
    y = y - 70;
    y = y * 2
```

```
const z = x + y;
console.log(z);

// Comparison Operators
// ==,===,!=,<,=<,>,>=

let x = 200;
let y = 400;

if (x > y) {
    console.log("X is bigger value", x)
}
else {
    console.log("Y is bigger value", y)
}

let value1 = 100;
const value2 = 100;
const value2 = 1000;
const value2 = "1000";
const value2 = true;
= , == , ===
if (value1 = value2) {  // assigneing the values
if (value1 == value2) { // check the values are same or not
if (value1 === value2) { // vaues and their data type
if (!(value1 === value2)) { // vaues and their data type
    console.log("values same")
}
else {
    console.log("values not same")
}

// Logical Operators(3)
// AND &&, OR ||, NOT!

let value1 = 1000; value2 = 2000; value3 = 400;

if ((value1 > value2) && (value1 > value3)) {
    console.log("Value 1 is bigger", value1)
```

```
}
else if ((value2 > value1) && (value2 > value3)) {
    console.log("Value 2 is bigger", value2)
}
else {
    console.log("Value 3 is bigger", value3)
}

if ((value1 > value2) || (value1 > value3)) {
    console.log("Value 1 is bigger", value1)
}
else if ((value2 > value1) || (value2 > value3)) {
    console.log("Value 2 is bigger", value2)
}
else {
    console.log("Value 3 is bigger", value3)
}



//typeOf operator
let ganapathi = 100;
let myname = "ganapathi";
let isvalidate = true;
let isvalidated = false;
let myvalue =

    function add() {
        return
    }

const hi = () => {
    return
}

let myarry = {
    hi: 100,
};
```

```
        let myObject = [1, 2, 4, 5];

        console.log(typeof (ganapathi));
        console.log(typeof (myname));
        console.log(typeof (isvalidate));
        console.log(typeof (isvalidated));
        console.log(typeof (myvalue));
        console.log(typeof (funame));
        console.log(typeof (add));
        console.log(typeof (hi));
        console.log(typeof (myarry));

        console.log(typeof (myObject));
        console.log(Array.isArray(myObject));

        //ternary operator ==> ?

        let ganapthi = 12;
        const voting = 18;
        // if (ganapthi >= voting) {
        //     console.log("eligible to vote");
        // }
        // else {
        //     console.log("not eligible to vote ");
        // }

        console.log(ganapthi >= voting ? "eligible to vote" : "not
eligible to vote");

    </script>
```

Statements ⇒ 2

Conditional statements (4)
Looping Statements (5)

https://pythontutor.com/visualize.html

Conditional statements (4)	Looping Statements (5)

if(){ Statements }; **if(){ Statements } else{ Statements};** **if(){ Statements }** **else if(){ Statements }** **else{ Statements};** **switch(){** **Case: “case”** **Break** **…** **Default:** **}**	**for(){ Statements };** **for(of) { Statements };** **for(in) { Statements };** **While () { Statements };** **do{ Statements } while();**

```
<script>
        //Conditional statements (4)
        // if(){ Statements };  ==> true (max) ==> success
        // let todaClass = true;
        let todaClass = false;
        if (todaClass) {
            alert("Yes, Definiy classes are there!.....")
        }


        let votingEligiblity = 18;
        // const ganapathi = 17;
        const ganapathi = 26;
        if (ganapathi > votingEligiblity) {
            alert("Eligible to voting ....!😍😍😍😍");
        }

        // if(){ Statements } ==> max (true, success) else{ Statements};
==> max (false, failure)
        if (todaClass) {
            alert("Yes, Definiy classes are there!.....")
        }
        else {
            alert("No,  classes today!.....")
        }
```

```
if (ganapathi > votingEligiblity) {
    alert("Eligible to voting ....!😍😍😍😍");
}
else {
    alert("Not Eligible to voting ....!❌❌🤦‍♂️🤦‍♂️");
}

// if(){ Statements }
// else if(){ Statements }
// else{ Statements};  ==> nested if statements ==> ladder

let a = 1000; b = 2000; c = 3000; d=4000;
// if (a > b) {
//     alert("A is bigger value..!")
// }
// else if (a > c) {
//     alert("A is bigger value..!")
// }
// else if (b > a) {
//     alert("B is bigger value..!")
// }
// else if (b > c) {
//     alert("B is bigger value..!");
// }
// else {
//     alert("C is bigger value..!")
// }

if ((a > b) && (a > c)) {
    alert("A is bigger value..!")
}
else if ((b > a) && (b > c)) {
    alert("B is bigger value..!")
}
else {
    alert("C is bigger value..!")
}
```

```
// switch(){
// Case: “case”
// Break
// …

// Default:
// }


// let userName = "superAdmin";
// let adminPass = "aa@123";

// if (userName == "superAdminmm") {
//     console.log("login completed...")
//     if (adminPass == "aa@123") {
//         console.log("login completed...");
//         console.log("Valid PAssword...");
//     }
//     else {
//         console.log("in Valid PAssword...");
//     }
// }
// else {
//     console.log("Can't login");
// }

// switch(){
// Case: “case”
// Break
// …

// Default:
// }

// arithmetic Operation

let a = 100; b = 200;
// let key = "add";
```

```
// let key = "sub";
let key = "divhjjhj";
switch (key) {
    case "add":
        let add = a + b;
        console.log("addition: ", add);
        break;

    case "sub":
        let sub = a - b;
        console.log("addition: ", sub);
        break;

    case "multi":
        let multi = a * b;
        console.log("addition: ", multi);
        break;

    case "div":
        let div = a / b;
        console.log("addition: ", div);
        break;
    case "modulo":
        let modulo = a % b;
        console.log("addition: ", modulo);
        break;
    case "Inc":
        a++;
        let Inc = a + b;
        console.log("addition: ", Inc);
        break;
    case "Dec":
        b--;
        let Dec = a - b;
        console.log("addition: ", Dec);
        break;

    default:
        console.log("You enter a wrong operation")
```

```
                break;

            // finally:
        }



    </script>
```

```
<script>
        // Looping Statements (5)
        //init
        //condition checking
        //inc/ dec

        // for(){ Statements };
        // for (let i = 0; i <= 5; i++) {
        //     console.log(i);
        // }
        // let i = 0;
        for (let i = 0; i <= 5; i++) {
        //     console.log(i);
        // }

        // let i = 0 ==> 0<=5 ==> console.log(i); ==> 0++ = 1
        // let i = 1 ==> 1<=5 ==> console.log(i); ==> 1++ = 2

        // let i = 4 ==> 4<=5 ==> console.log(i); ==> 4++ = 5
        // let i = 5 ==> 5<=5 ==> console.log(i); ==> 5++ =6
        // let i = 6 ==> 6<=5 ==> x


        // for(of) { Statements };

        let myArr = [10000, 2, 3, 4, 5, 6, 7];
        console.log(myArr[0]);
        console.log(myArr[1]);
        console.log(myArr[3]);
```

```
        for (index of myArr) {
            console.log(index)
        }

        // for(in) { Statements };

        let myObj = {
            name: "ganapathi",
            mobile: 7502132895,
            gmail: "vgr@gmail.com"
        };
        console.log(myObj);
        console.log(myObj.name);
        console.log(myObj.mobile);
        console.log(myObj.gmail);

        for (keys in myObj) {
            console.log(`my object value ${keys} and values
${myObj[keys]}`)
        }


        // While () { Statements };
        let i = 1;
        while (i < 6) {
            console.log("while loop", i);
            i++;
        }

        // let i = 1
        // 1<=6;
        // console.log("while loop", i);  ==> 1 ++

        // do{ Statements } while();

        let i = 0;
        do {
```

```
            console.log("while loop", i);
            i++;
        } while (i <= 5);


    </script>
```

INbuilt Object Function

Math object (5)

Date Object (5 to 8)

```
<script>
        // INbuilt Object Function
        // Math object (5)
        let num1 = 10.47;
        let num2 = 10.50;
        let num3 = 10.40;
        let num4 = 10.56;
        let num5 = 16;
        let num6 = 2;



        console.log(Math.abs(num1))
        console.log(Math.round(num1))
        console.log(Math.round(num4))
        console.log(Math.ceil(num3))
        console.log(Math.floor(num4))
        console.log(Math.sqrt(num5))
        console.log(Math.sqrt(num6))
        console.log(Math.pow(2, 4))
        console.log(Math.log(10))
        console.log(Math.max())
        console.log(Math.min())


        // console.log(num1);
        // console.log(num2);
        // console.log(num3);
        // console.log(num4);
```

```
        // console.log(num5);
        // console.log(typeof (num1));
        // console.log(typeof (num2));
        // console.log(typeof (num3));
        // console.log(typeof (num4));
        // console.log(typeof (num5));

        // Date Object (5 to 8)

        let myDate = new Date();
        console.log(myDate);
        console.log(myDate.getDate());
        console.log(myDate.getDay());
        console.log(myDate.getMonth());
        console.log(myDate.getFullYear());
        console.log(myDate.getHours());
        console.log(myDate.getMinutes());
        console.log(myDate.getMilliseconds());
        console.log(myDate.getSeconds());
        console.log(myDate.getTime())
        console.log(myDate.toString())
        console.log(myDate.toDateString())

    </script>
```

Operation

- **String OPerations (5 to 8)**
- **Array Operation (28 ⇒ 15)**

DOM Manipulating(5) & Selecting Elements (5)

document.getElementById()

Js I/O Methods

- **Input method ⇒ 2**
- **Output Method ⇒ 4**

Validation of Static Web Pages

ES 6 Topics